Theory of Computation

DAYANANDA SAGAR
UNIVERSITY

Dr. Shreyas Rajendra Hole **Prof. SriramKumar R.**

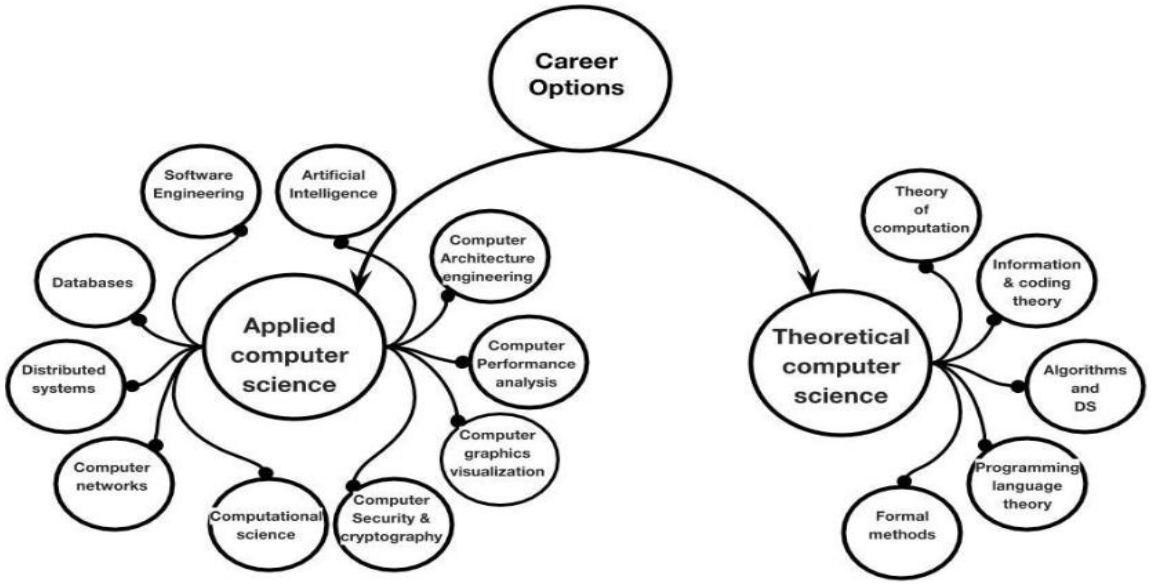
Shreyasrajendra.hole-aiml@dsu.edu.in

Theory of Computation

DAYANANDA SAGAR UNIVERSITY

Theory of computation is the branch that deals with how efficiently problems can be solved on a model of computation , using an algorithm.

- Applied Computer Science deals with practical implementation and the application of computer science principles to solve real-world problems.
- This field involves using existing theories, algorithms, and technologies to develop software, hardware,



- Theoretical Computer Science focuses on understanding the fundamental aspects of computation.
- It involves creating models, developing algorithms, and exploring the limits of what can be computed.

1. **Focus**: Applied Computer Science **is practical and solution-oriented**, while Theoretical Computer Science **is abstract and focuses on fundamental principles.**
2. **Methods**: Applied Computer Science **uses existing theories to develop applications,** whereas Theoretical Computer Science **develops new theories and models**.
3. **Outcomes**: Applied Computer Science **produces software, systems, and applications,** while Theoretical Computer Science **produces new algorithms, proofs, and computational models.**

DAYANANDA SAGAR UNIVERSITY

V SEM – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

V SEMESTER												
SL	Course Type	Course Name	Teaching Department	Teaching Hours / Week				Examination				Credits
				Lecture	Tutorial	Practical	Project	Duration in Hours	CIE Marks	SEE Marks	Total Marks	
				L	T	P	J					
1	IPCC / PCC	Machine Learning		3	0	2	0	05	60	40	100	**04**
2	IPCC / PCC	Operating Systems		3	0	2	0	05	60	40	100	**04**
3	IPCC / PCC	Theory of Computation		3	0	0	0	03	60	40	100	**03**
4	IPCC / PCC	Computer Networks		3	0	2	0	05	60	40	100	**04**
5	SEC	Skill Enhancement Course - III (Web Development)		1	0	2	0	01	100	--	100	**02**
6	PEC	Professional Elective Course -1		3	0	0	0	03	60	40	100	**03**
7	OEC	Open Elective – I		3	0	0	0	03	60	40	100	**03**
		Total		19	0	8	0					**23**

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Course Learning Objectives:

This course will enable students to:

1. Understand the concept of finite automata.
2. Utilize regular expressions in practical applications like search and data extraction.
3. Analyze Properties of Regular and Context-Free Languages
4. Explore Context-Free Grammars and Pushdown Automata
5. Explain the working principles of Turing Machines as computational models.
6. Analyze the concept of undecidability and its implications in computation

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

UNIT – I	**11 Hours**
Introduction to Finite Automata. The Central Concepts of Automata Theory, Deterministic Finite Automata, Non deterministic Finite Automata, An application, Finite Automata with Epsilon Transitions. ***Textbook 1: Chapter 1.1.1, 1.5,2.1,2.2,2.3,2.4,2.5***	
UNIT – II	**10 Hours**
Regular Expression and Languages: Regular Expressions, Finite Automata and Regular Expressions, Applications of Regular Expressions, Properties of Regular Languages: Pumping Lemma for Regular Languages, Closure properties of Regular Languages, Equivalence and Minimization of Automata ***Textbook 1: Chapter 3.1,3.2,3.3,4.1,4.2,4.3,4.4***	
UNIT – III	**11 Hours**
Context Free Grammars and Languages: Context Free Grammars, Parse Tree, Applications of Context Free Grammar, Ambiguity in Grammars and Languages. Definition of Pushdown Automata, The Languages of a PDA, Equivalence of PDA's and CFG's, ***Textbook 1:Chapter 5.1,5.2,5.3,5.4,6.1,6.2,6.3***	
UNIT – IV	**10 Hours**
Deterministic Pushdown Automata, Properties of Context Free Languages, The Pumping Lemma for Context Free Languages, Closure Properties of Context Free Languages. ***Textbook 1: Chapter 6.4,7.1,7.2,7.3***	
UNIT – V	**10 Hours**
Introduction to Turing Machines: The Turing Machines, Undecidability A language that is not Recursively Enumerable, An Undecidable Problem That is RE, Post Correspondence Problem, Other Undecidable Problems, Textbook 1: Chapter 8.1,8.2,9.1,9.2,9.4,9.5	

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

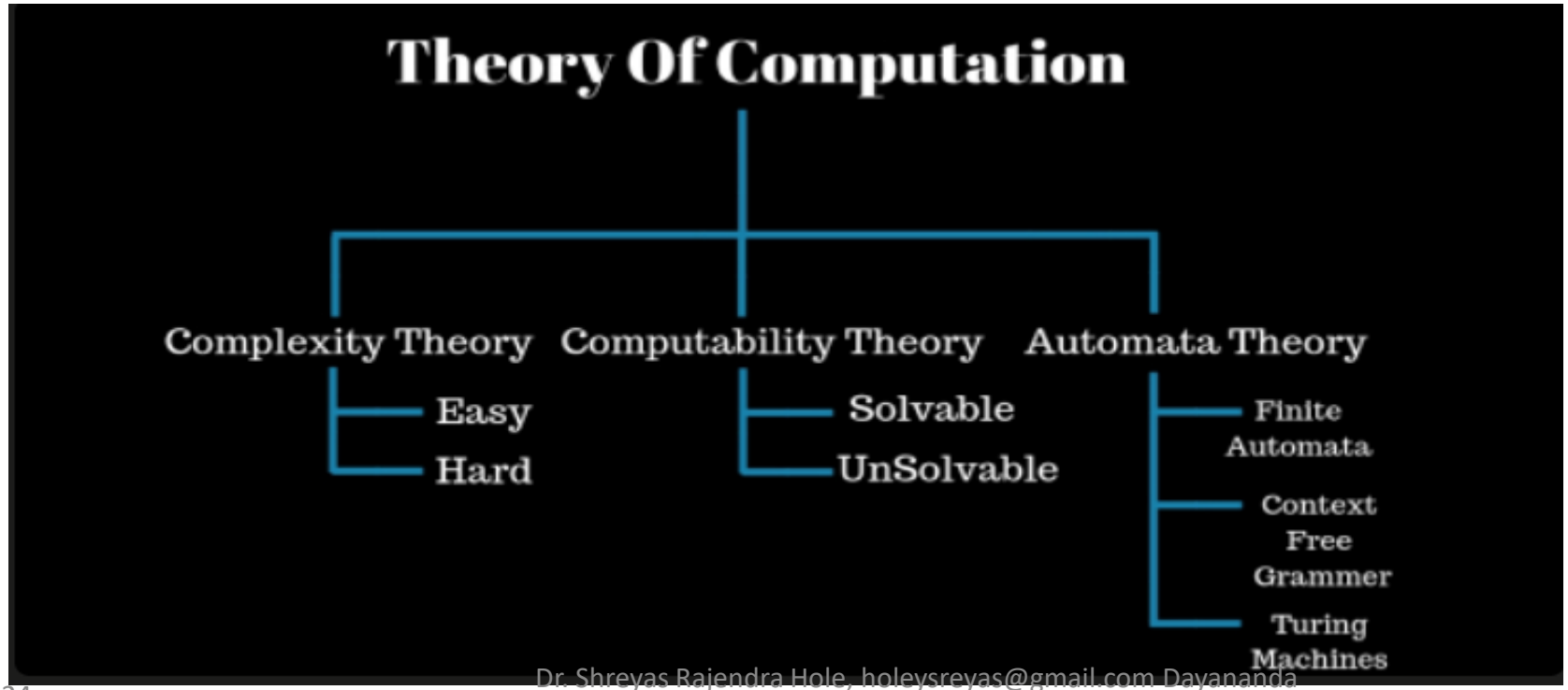
UNIT I

DAYANANDA SAGAR UNIVERSITY

1. Introduction to Finite Automata.
2. The Central Concepts of Automata Theory,
3. Deterministic Finite Automata,
4. Non deterministic Finite Automata,
5. An application,
6. Finite Automata with Epsilon Transitions.

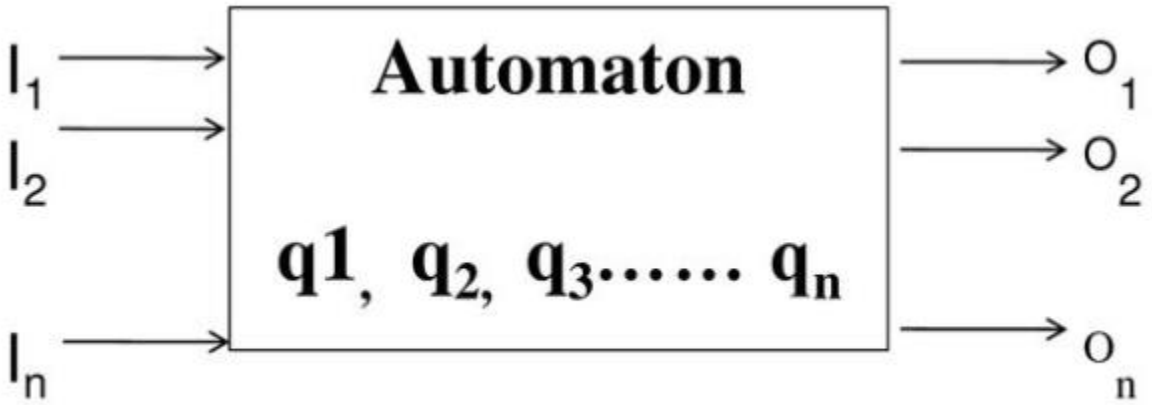
Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

- **The Theory of Computation is a fundamental subject in Computer Science and Engineering that deals with the study of computation and computational complexity.**
- **It involves the analysis of algorithms, data structures, and computational models to understand the efficiency and feasibility of solving computational problems.**
- **The subject is divided into three main branches: Automata Theory, Computability Theory, and Computational Complexity Theory.**



- What is Automaton?

  An automaton is a system where energy, material and information are transformed, transmitted and used for performing some function without direct participation of man.



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Automata Theory:

•**Definition**: Automata Theory is the study of abstract machines (or more appropriately, abstract 'mathematical' machines or systems) and the computational problems that can be solved using these machines.

Key Concepts:

•**Finite Automata**: A finite automaton is a mathematical model that can be in one of a finite number of states. It can change its state based on the input it receives.

•**Regular Languages**: Regular languages are a class of formal languages that can be recognized by a finite automaton.

•**Context-Free Grammars**: Context-free grammars are a type of formal grammar that can generate context-free languages.

Computability Theory:

•**Definition**: Computability Theory is the study of the limitations of computation. It examines whether a problem can be solved using a computer and if so, how efficiently it can be solved.

Key Concepts :

Turing Machines: Turing machines are a model of computation that can solve any problem that can be solved by a computer.

•**Halting Problem**: The halting problem is a famous problem in computability theory that asks whether there exists a program that can determine whether another program will run forever or eventually stop.

•**Undecidability**: Undecidability is a concept in computability theory that states that there are problems that cannot be solved by a computer.

Computational Complexity Theory:

•**Definition**: Computational Complexity Theory is the study of the resources required to solve computational problems. It examines the time and space complexity of algorithms.

Key Concepts:

Time Complexity: Time complexity is a measure of how long an algorithm takes to complete.

•**Space Complexity**: Space complexity is a measure of how much memory an algorithm requires.

•**NP-Completeness:** NP-completeness is a concept in computational complexity theory that states that a problem is difficult to solve efficiently if it is reducible to another known NP-complete problem.

- The term '**automata**' is derived from the Greek word, meaning '**self-acting, self-willed, self-moving**'.
- **Automata Theory is a foundational area in theoretical computer science that studies abstract machines and the problems they can solve.**

DAYANANDA SAGAR UNIVERSITY

1. **Automata Theory is the study of abstract computational devices or machines and the computational problems that can be solved using these devices.**
2. **It is a branch of theoretical computer science that deals with the design and analysis of algorithms for automating computational processes.**
3. **Automata Theory examines the logical structure of discrete systems and the rules governing their behavior and state transitions.**
4. **It is the mathematical study of machines or automata that perform computations based on a set of given rules or instructions.**
5. **Automata Theory explores the capabilities and limitations of different types of computational models, such as finite automata, pushdown automata, and Turing machines.**

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

1. Automata Theory provides a formal basis for the design of digital circuits, language processors, and various types of software and hardware systems.
2. It investigates the formal properties of languages and grammars, particularly the classification and recognition of different types of formal languages.
3. Automata Theory studies the algorithms and computational processes that can be represented by state machines with discrete inputs and outputs.
4. It is a field that explores how abstract machines can simulate different types of computation, from simple pattern matching to complex problem solving.
5. Automata Theory involves the analysis of state-based systems and their transitions, focusing on the mathematical representation of these systems.

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

1. Automata Theory examines the expressive power of different computational models and how they relate to each other in terms of computational complexity.
2. It is the study of formal languages, automata, and the computational problems related to language recognition and generation.
3. Automata Theory is a foundational discipline that underpins many areas of computer science, including compiler design, artificial intelligence, and the theory of computation.
4. It is the theoretical framework for understanding the computational aspects of mechanical devices and how they process information.
5. It provides tools and techniques for modeling and analyzing the behavior of systems that evolve over time according to a set of predefined rules.

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

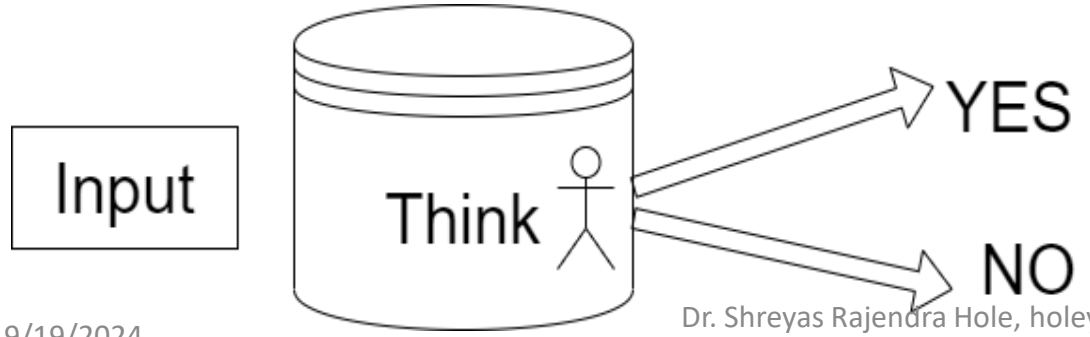
Example 1: Design a machine who identify Binary strings end with 0

11001100 =0

=1

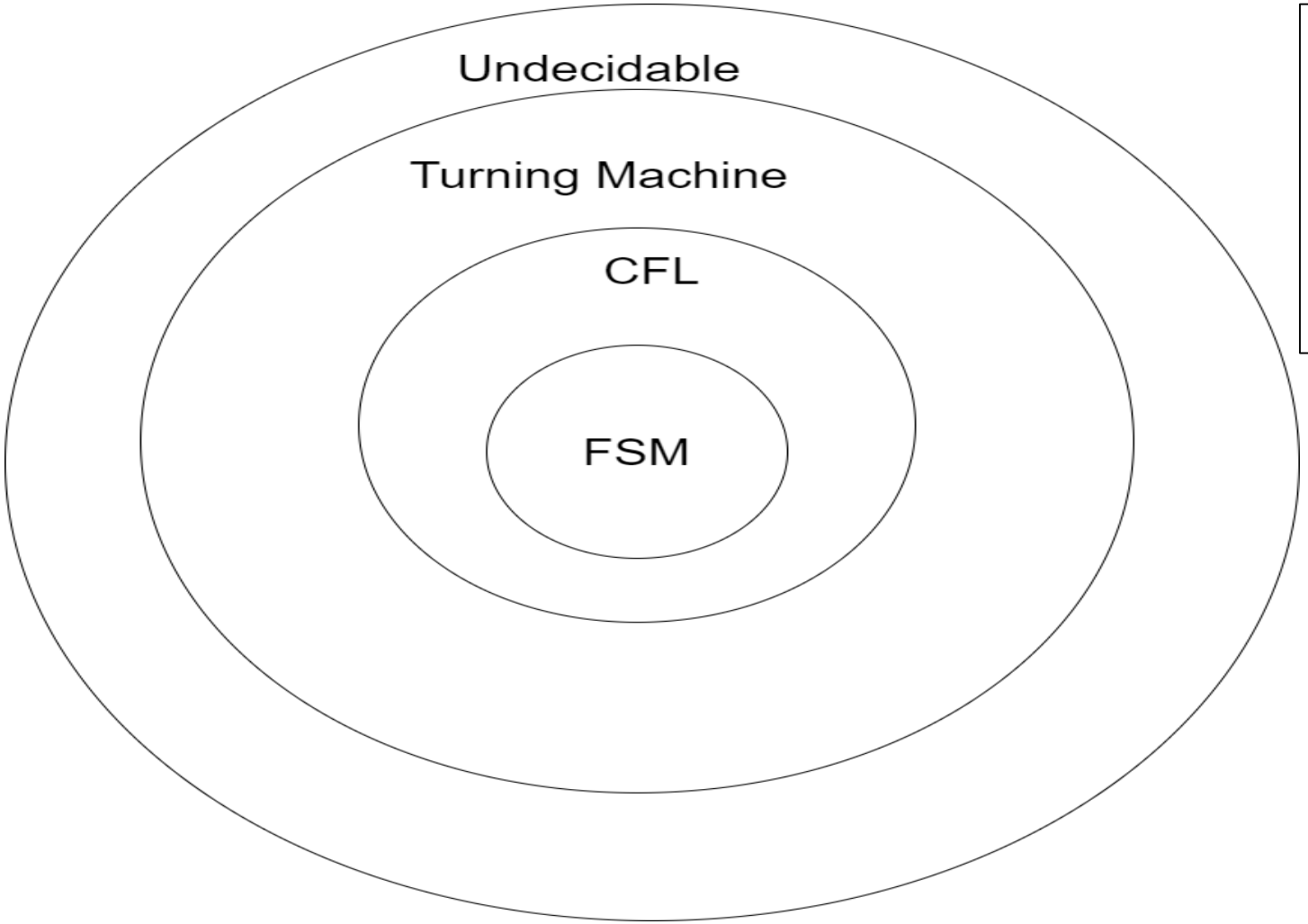
Example 2:Design a machine accepts all valid java codes?
Is this possible to designed valid binary data code????

Example 3: Accepts all valid java code and never goes into infinite loop?



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

There are many layers in TOC



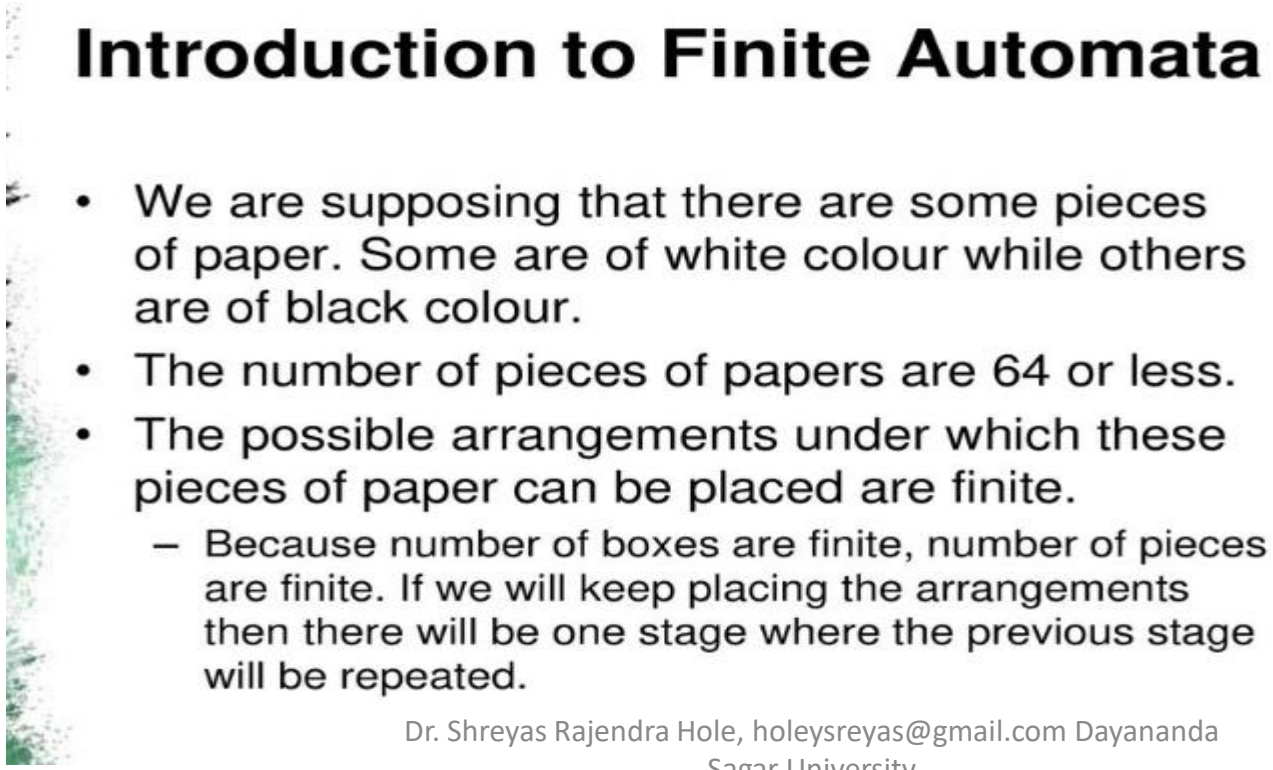
- **FSM: Finite state Machine**
- **CFL: Context Free language**
- **Turning Machine**
- **Undecidable**

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

1.1 Introduction to Finite Automata.

DAYANANDA SAGAR UNIVERSITY

- Finite automata are abstract computing devices used to recognize patterns and analyze language expressions.
- They consist of a finite set of states, an input alphabet, a transition function that determines the next state based on the current state and input symbol, an initial state, and a set of final states or accepting states.



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Finite automata are used in various real-world applications, including:

- **Automatic Doors**: Finite automata are used in automatic doors to manage the opening and closing of doors based on the presence or absence of people. The door's state changes depending on the input (person present or absent) and follows a predetermined sequence of states

- **Vending Machines**: Vending machines can be modeled using finite automata to manage the selection and dispensing of products based on user inputs (coin insertion, button presses)

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

FSM: Finite State Machine Basics

- **Symbol** : 0,1,2,3,a,b,c,d
- **Alphabet**: Collection of Symbol

  $\sum\{a,b,c,d\}$

  $\sum\{0,1,2,3\}$

  $\sum(Sigma)$
- **String** : Sequence of symbols

Eg: a,b,c,0,1

aa, bb, 01,

- **Language:** Set of Strings $\sum = \{0,1\}$

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

- L1= Set of all strings of Length 2 (0,1)
  ={ }
- L2= Set of all strings of length 3 (0,1)
  ={ }

L3= Set of all strings that begin with 0
= { }

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Power of Sigma $\sum(Sigma)$

Power of Σ. $\sum^0 = \{\varepsilon\}$

E= Epsilon
= Empty strings

$\sum^0$ =set of all strings of length 0 $\sum^0 = \{\varepsilon\}$

$\sum^1$ =set of all strings of length 1 $\sum^1 = \{0,1\}$

$\sum^2$ =set of all strings of length 2 $\sum^2 = \{00,01,10,11\}$

$\sum^3$ =set of all strings of length 3 $\sum^2 = \{?\}$

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Cardinality: Number of element in a set

$$\sum^0 = \{\varepsilon\}$$ So cardinality is 1

$$\sum^1 = \{0,1\}$$ Cardinality is 2 and so on…

$$\sum^n =$$ Set of all strings of length n

If it is not number between 0 to 1 then

$$\sum^n = 2^n$$ is cardinality

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Sigma *

$$\sum^* = \sum^0 \cup \sum^1 \cup \sum^2 \cup \sum^3 \ldots\ldots\ldots$$

$$\sum^* = \{\varepsilon\} \cup \{0,1\} \cup \{00,01,10,11\} \cup \ldots\ldots\ldots\ldots\ldots\ldots$$

= Set of all possible strings of all lengths over {0,1}

Guess it is the Finite or Infinite strings ????

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

FINITE STATE MACHINE (FSM)

or

FINITE AUTOMATA (FA)

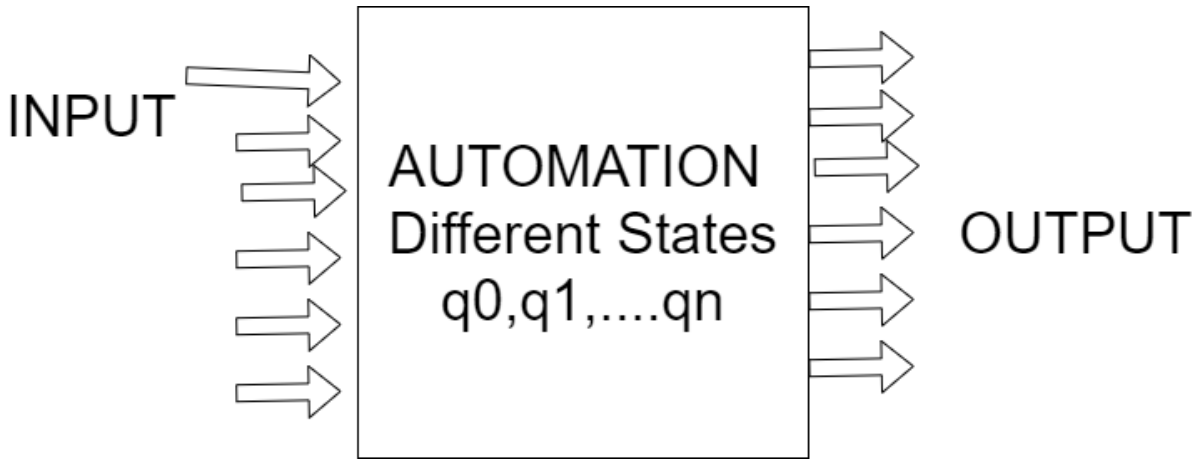
DAYANANDA SAGAR UNIVERSITY

Finite State Machine

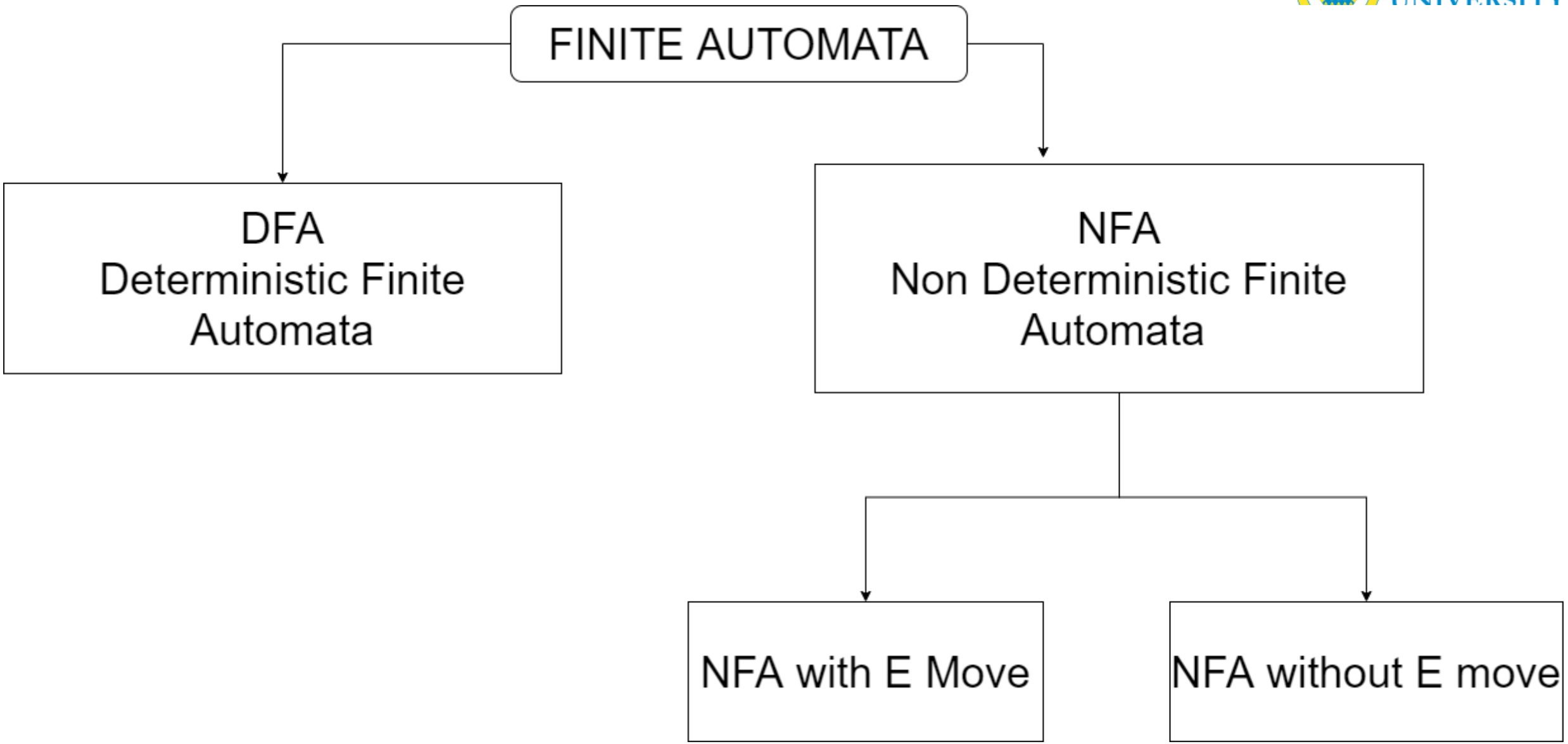
A finite state machine (FSM) or Anite Automata (FA) is a mathematical model of a symbol with discrete input and output. The system can have any one of a finite number of internal configuration or state.

A Finite Automata (FA) consist of finite set of states and a set of teansitions from state to state that occur an input symbol choosen from an alphabet Σ.

One state usually denoted by q_0 is the initial state, in which the automation starts. Some states are designated as final states or accepting states



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

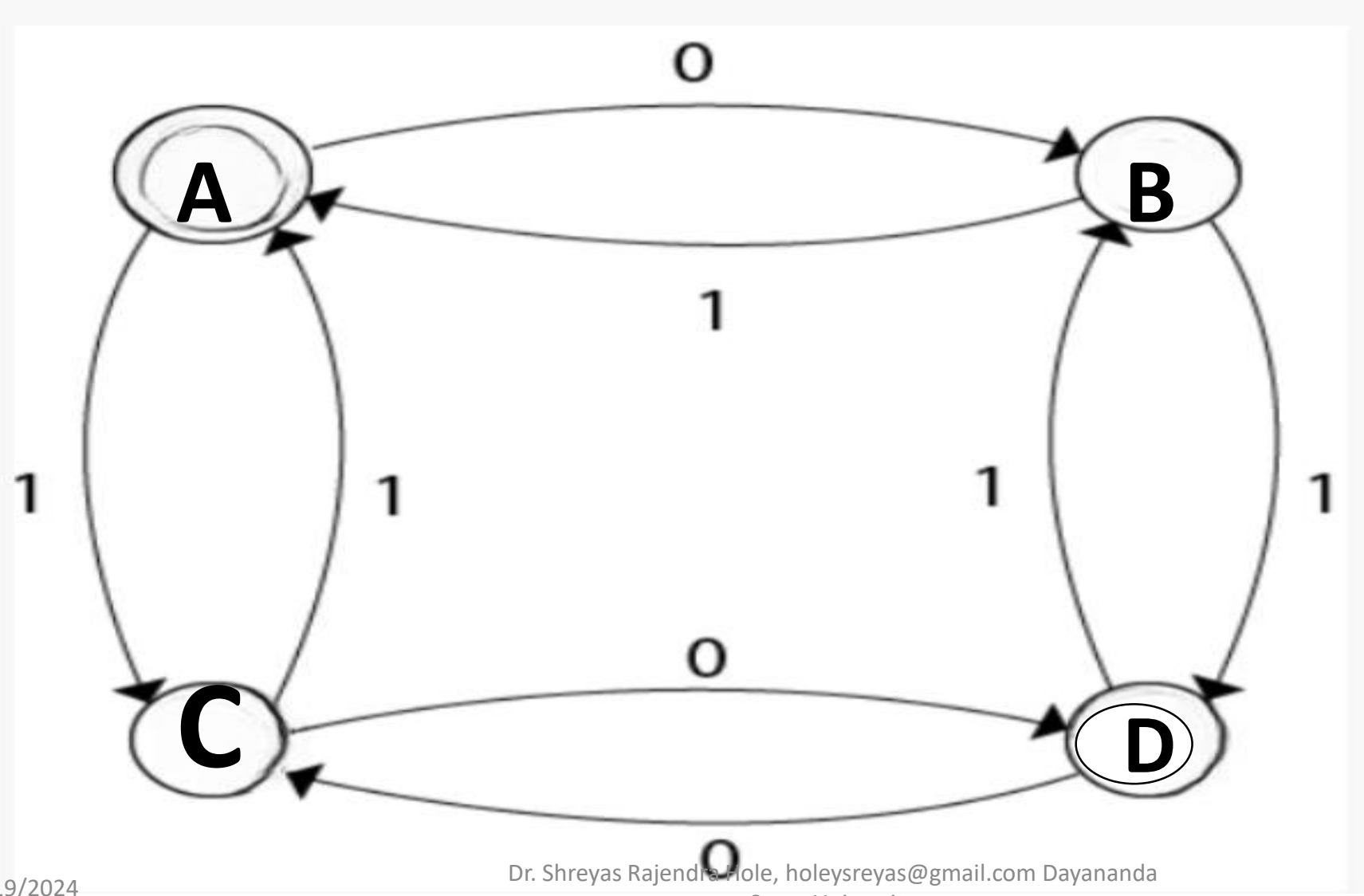
The key features of **finite automata** include:

1. **Input and output**: Finite automata take a string of symbols as input and either accept or reject the input based on whether it reaches an accepting state.

2. **States and transitions**: Finite automata have a finite set of states, and the transition function determines how the automaton moves from one state to another based on the current state and input symbol.
3. **Formal specification**: Finite automata are formally defined as a 5-tuple (Q, Σ, δ, q0, F), where

- **Q** is the set of states,
- **Σ** is the input alphabet,
- **δ** is the transition function,
- **q0** is the initial state, and
- **F** is the set of final or accepting states.

DFA Deterministic Finite Automata



The directed graph is called transition diagram is associated with FA.

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

DAYANANDA SAGAR UNIVERSITY

Example 1 :

The Finite state machine (M) Given in Following table find out the strings among the following string which are accepted by Machine (M)

- **101101**
- **11111**
- **000000**

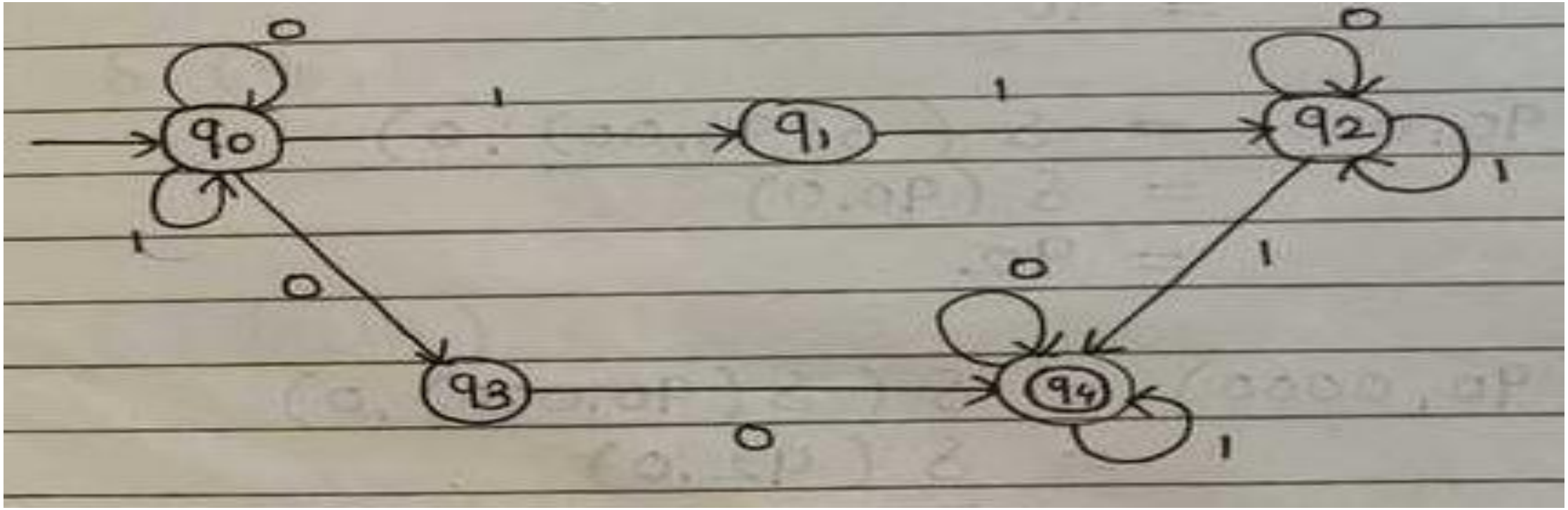
Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Tutorial
Example 01

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

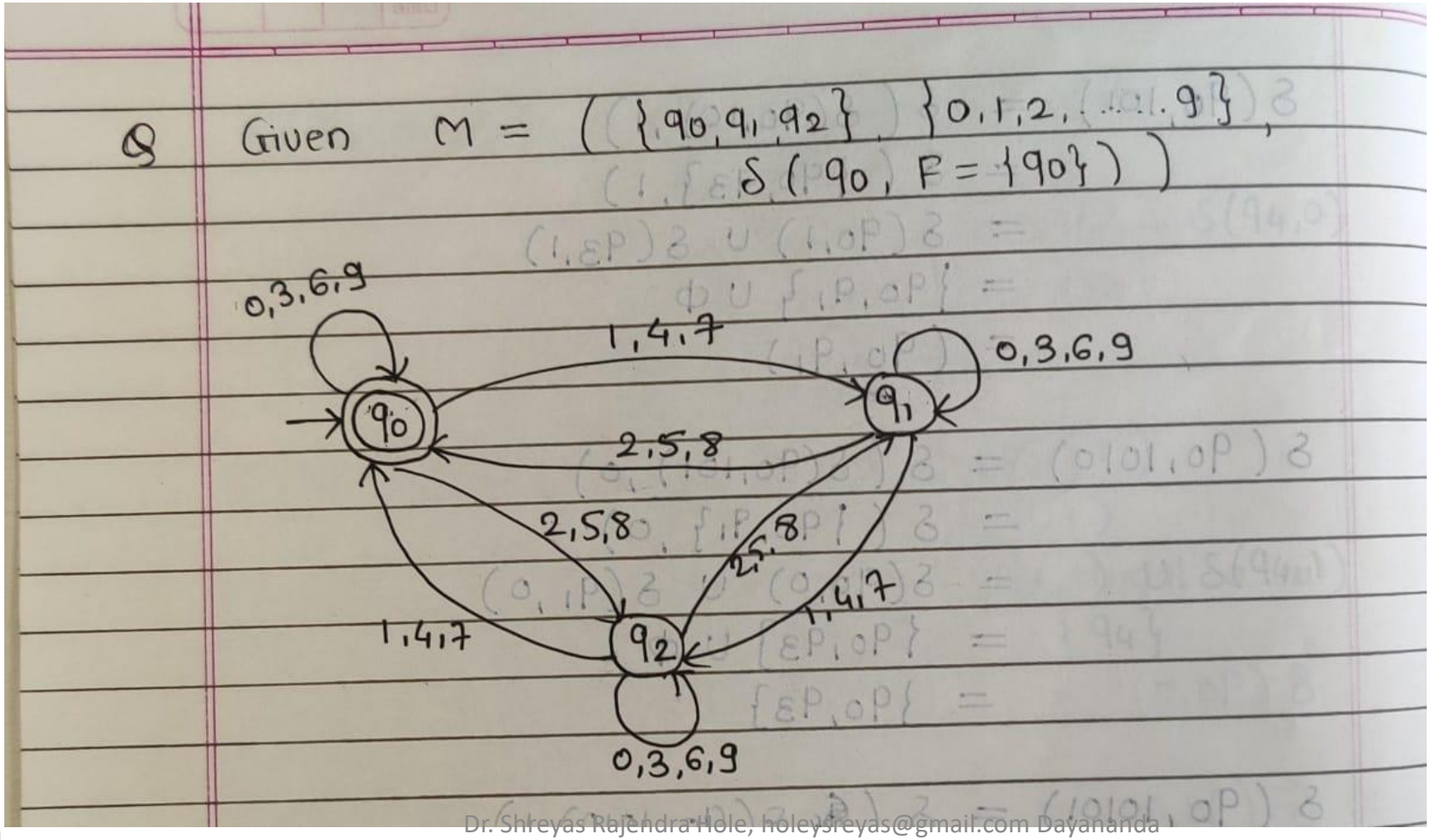
Example 2 :

For the transaction system M in given figure. Obtain the sequence of states for the input 000101 also find an input 000101. also find an input sequence not accepted by M. Also consider string 101011



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 03: Given M=({qo,q1,q2} {0,1,2,….9} δ (qo,F={90}))



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

DFA: Deterministic Finite Automata

Example 01:

Design DFA Machine

L1= Set of all Strings that start with 0

L1 is a language; Design DFA for L1

={ }

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

DAYANANDA SAGAR UNIVERSITY

Example 02:
Construct a DFA that accepts of all strings over {0,1} of length 02

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

DAYANANDA SAGAR UNIVERSITY

DFA: Deterministic Finite Automata

Example 01: Construct DFA over $\sum = \{0,1\}$ with ending 00

Find:

1. State Diagram / Transition Graph/ Transition Diagram (2)
2. Tuple (2)
3. Transition Function table (2)
4. Verification (Consider for Acceptance& Rejection Strings) (4)

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 03: Design and construct DFA that accepts any strings over {a,b} that contain strings aabb in it

Given : $\sum = \{a, b\}$

1. **State Diagram / Transition Graph/ Transition Diagram (2)**
2. **Tuple (2)**
3. **Transition Function table (2)**
4. **Verification (Consider for Acceptance& Rejection Strings) (4)**

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 04: Construct DFA over $\sum = \{0,1\}$ ending with 00

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

DAYANANDA SAGAR UNIVERSITY

Tutorial 02

Design a Finite State Machine (FSM) using a Deterministic Finite Automaton (DFA) that represents your mobile lock system. The FSM should only accept strings that match your password, which must be at least 5 characters long. Your task is to:

- **Define the states and transitions.**
- **Illustrate the FSM with a state diagram.**
- **Clearly indicate the start state and accept state(s).**
- **Ensure the DFA correctly validates your password.**

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 05: Construct DFA all strings on {0,1} Containing 101 as its Substring.

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 06: Construct DFA which accepts all the strings containing 00 as substring over $\sum^*$ such that $\sum = \{0,1\}$

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 07: Construct DFA which accepts all the strings ending with 101 over $\sum^*$ such that $\sum = \{0,1\}$

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 08: Construct DFA which start with 00 over

$$\sum = \{0,1\}$$

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 09: Design a DFA for strings which accept set of all strings on $\sum = \{0,1\}$ starting with prefix ‘ab’

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 010: Construct DFA which accepts set of all strings on $\sum = \{0,1\}$ start with a & end with a.

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

DAYANANDA SAGAR UNIVERSITY

Example 011: Construct DFA which accepts all the strings ending with 101 over $\sum^*$ such that $\sum = \{0,1\}$

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

DAYANANDA SAGAR UNIVERSITY

Example 012: Construct DFA which accepts all the strings ending with 101 over $\sum^*$ such that $\sum = \{0,1\}$

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 013: Construct DFA which accepts all the strings starts with & ending with "a" over $\sum = \{0,1\}$ such that

1. **State Diagram / Transition Graph/ Transition Diagram (2)**
2. **Tuple (2)**
3. **Transition Function table (2)**
4. **Verification (Consider for Acceptance& Rejection Strings) (4)**

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 14:

Design a DFA that Accepts set of strings containing exactly 1 over alphabet Σ={0,1}

1. **State Diagram / Transition Graph/ Transition Diagram (2)**
2. **Tuple (2)**
3. **Transition Function table (2)**
4. **Verification (Consider for Acceptance& Rejection Strings) (4)**

1.2 The Central Concepts of Automata Theory

DAYANANDA SAGAR UNIVERSITY

The basic terminologies/concepts, which are important and frequently used in Formal Languages and Automata Theory.

1. **Symbol**: **A symbol is the smallest building block (an abstract entity), which can be any alphabet, digit, or any picture.**

Example: Frequently used symbols are: a, b, c, ……, z, A, B,…,Z, 0, 1, …..9

2. Alphabet: **An alphabet is a finite, nonempty set of symbols. It is denoted by Σ(Sigma). Example: Frequently used alphabets are:**

Σ = {0,1} is an alphabet of binary digits

Σ = {0,1,2,….9} is an alphabet of decimal digits

Σ = {a,b,c} is one alphabet

Σ = {A,B,C,….,Z} is an alphabet of Capital Letters

Σ = {0,1,a,b} is one alphabet

Σ = {a,b,c,#} is one alphabet

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

3. String: A string is a finite sequence of symbols chosen from some alphabet. It is generally denoted by w OR s and string is also called a word.

Examples:

- abaaab is a strings from the alphabet Σ={a,b}
- 01, 000, 1111, and 01101 are the strings from the binary alphabet Σ={0,1}

4. Length of a string:

The length of a string is the number of symbols in the string. It is denoted by |w|.

Example:

If w=abaa is a string then length is |w|=|abaa|=4.

5. Substring:

A part of a string is called a substring.

Ex: Let a string w=abcd , then Substrings are: a , b , c , d , ab , bc , cd , abc , bcd , and abcd.

But acd is not a substring. Similarly ac , ad , bd , abd are not substrings.

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

6. Prefix of a string (or) Starting substring:

A prefix of a string is any number of leading symbols of that string.

Ex: Let a string w=abc ,

then Prefixes are: ε , a , ab and abc

7. Suffix of a string (or) Ending substring:

A suffix of a string is any number of trailing symbols of that string.

Ex: Let a string w=abc ,

then Suffixes are: ε , c , bc and abc

8. Powers of an alphabet (Powers of Σ):

Let Σ be an alphabet, then ΣK is nothing but set of all strings of length K, over Σ.

If Σ={a,b}

- Σ1 is nothing but set of all strings of length '1' over Σ

Σ1 ={a,b} and total strings is: |Σ1| = 2

- Σ2 is nothing but set of all strings of length '2' over Σ

Σ2 => Σ.Σ (concatenation) => {aa, ab, ba, bb} and total strings is: |Σ2| = 4

9. Empty String (ε):

The empty string consisting of zero symbols, it is denoted by ε(epsilon).

Σ0 is nothing but set of all strings of length ‘0’ over Σ

Σ0 ={ε} , length of ε is |ε|=0, and total strings is: |Σ0| = 1

10. Kleene Closure (Σ*):

Set of all possible strings including epsilon(ε) over an alphabet Σ is called kleene closure and it is denoted with Σ*.

Let Σ={a,b}.

Σ* = Σ0 U Σ1 U Σ2 U Σ3 …….. Σn i.e Union of all lengths of strings.

Σ* = {{ε} U {a,b} U {aa,ab,ba,bb} ……} i.e All Possible Strings over Σ

Σ* = {ε, a, b, aa, ab, ba, bb ……}

11. Positive Kleene Closure (Σ+):

Set of all possible strings excluding epsilon(ε) over an alphabet Σ={a,b}.

Σ+ = Σ1 U Σ2 U Σ3 …….. Σn

Σ+ = {{a,b} U {aa,ab,ba,bb} ……}

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

12. Language:

A set of strings, all of which are chosen from some Σ^* is called a language.It is denoted by L. Where Σ is a particular alphabet.

Or

we can say A Language is a subset of Σ^*.

- Σ^* is the super set of all the languages i.e If L is a language, then $L \subseteq \Sigma^*$

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

NFA

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

A Nondeterministic Finite Automaton (NFA) is a type of finite state machine used in the field of automata theory and formal languages. It is a mathematical model of computation that consists of:

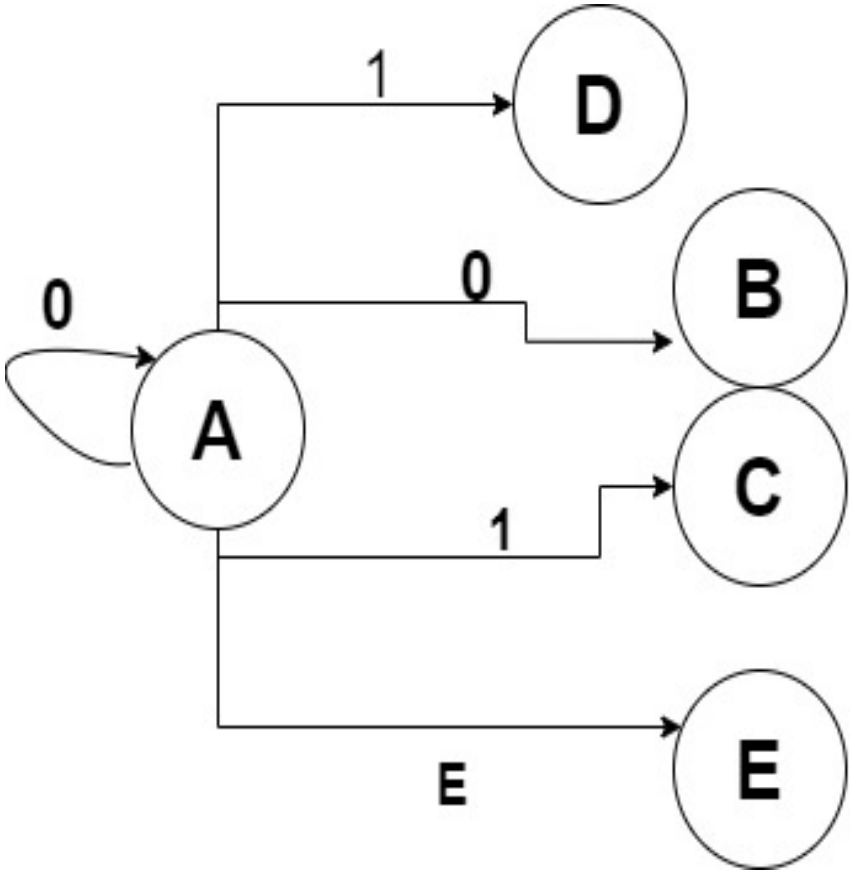
- States: A finite set of states.
- Alphabet: A finite set of input symbols.
- Transition function: A set of rules or transitions that define how the automaton moves between states.
- Initial state: The starting state from which the NFA begins.
- Accepting states: A set of states that represent accepting or final conditions.

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

The key feature of an NFA is its **nondeterminism**, meaning:

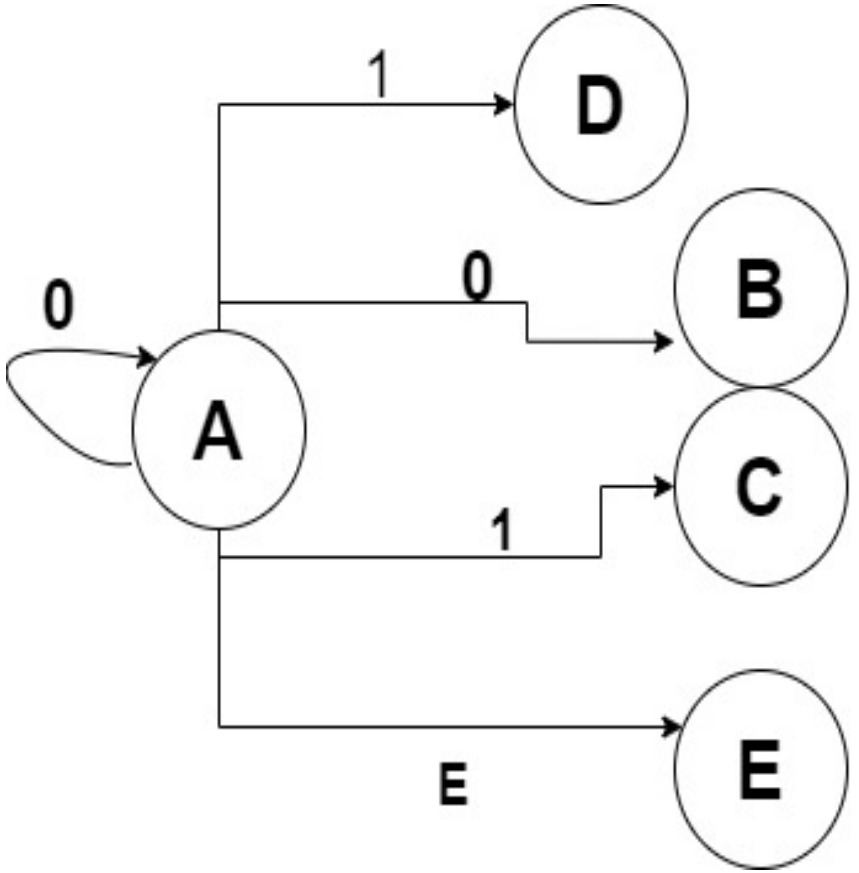
•At each state, for a given input symbol, the NFA can move to **multiple possible states** or even stay in the same state.

•It may also include transitions that do not require any input symbol, known as **epsilon (ε) transitions**.

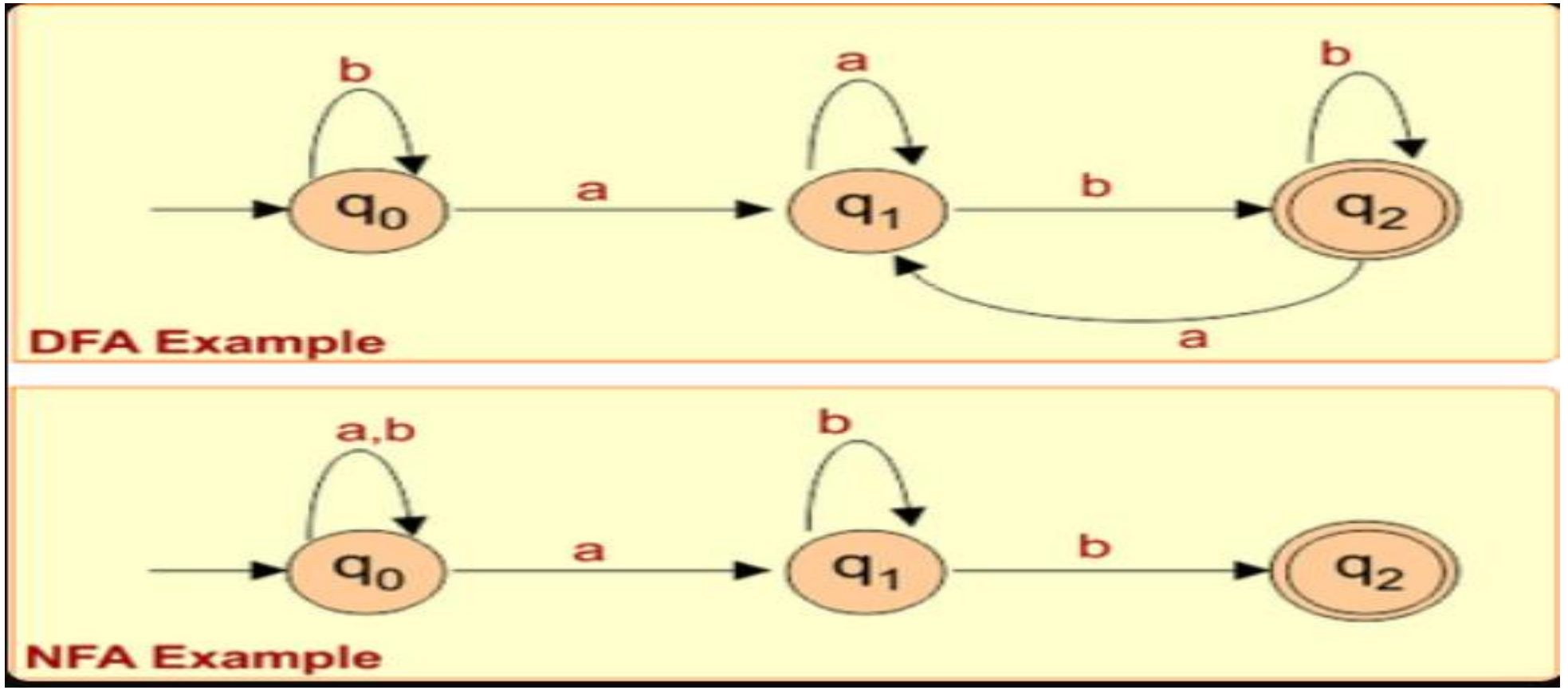


Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

- **In NFA , given the current State there cold be multiple next states.**
- **The Next state nay be chosen may be at random.**
- **All the next may be chosen at random.**



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

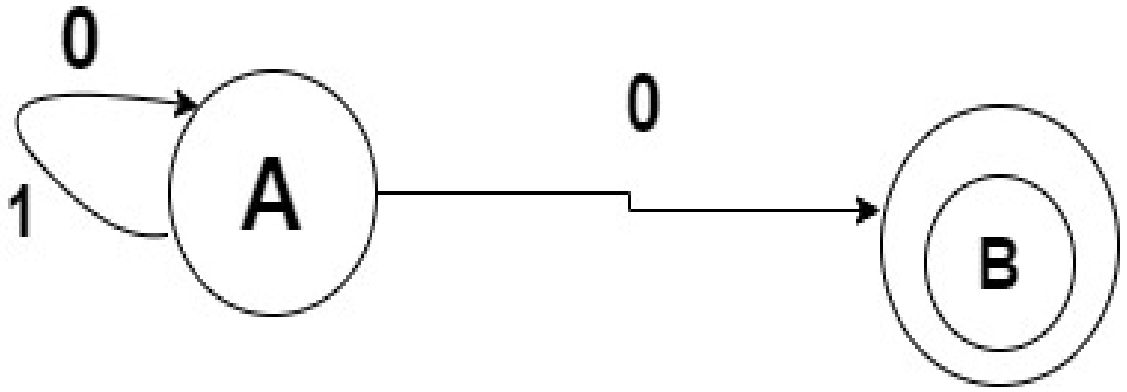
Feature	DFA (Deterministic Finite Automata)	NFA (Nondeterministic Finite Automata)
Transition Function	Exactly one transition for each state and input symbol.	Zero, one, or multiple transitions for each state and input symbol.
Epsilon (ε) Transitions	Not allowed.	Allowed (transitions without consuming input).
Backtracking	Not required; deterministic behavior.	May require backtracking to explore multiple paths.
Complexity of Construction	More complex to construct.	Simpler to construct.
Computation Model	Single machine processing input.	Multiple paths can be processed simultaneously.
Acceptance Criteria	Accepts if it ends in an accepting state.	Accepts if at least one path ends in an accepting state.
Space Requirement	Generally requires more space.	Generally requires less space.
Conversion from Regular Expressions	More complex.	Simpler and more straightforward.
Efficiency	More efficient in terms of time and space during execution.	Less efficient due to multiple possible paths.

$$\delta = Q \times \sum \rightarrow Q$$

$$\delta = Q \times \sum \rightarrow 2^Q$$

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example 01: L={Set of all the strings that ends with 0}



State	Input	
	0	1
A	AB	A
B	ΦNil	ΦNil

Calculate Tuple:
Q,
Σ,
Initial State,
F,
δ

$$\delta = Q \times \sum \rightarrow 2^Q$$

Q=2 States; Possibilities=04
A-----A,B,AB,Φ

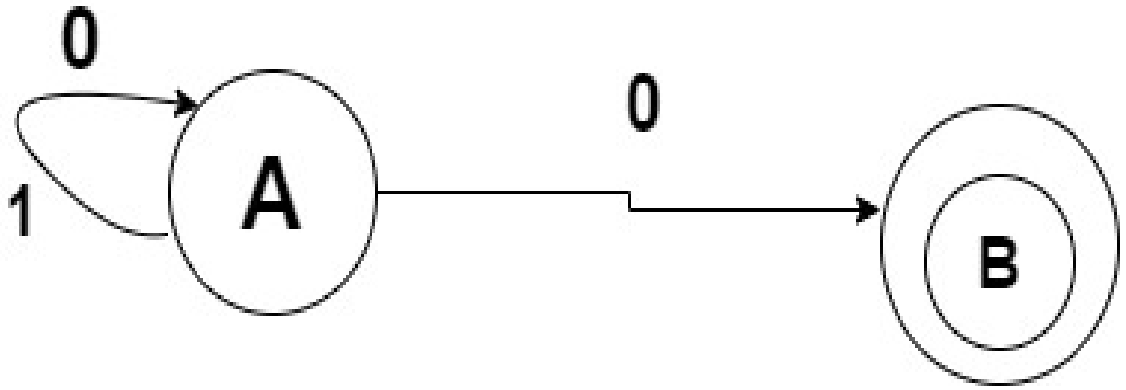
Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com, Dayananda Sagar University

03 State= 8 Possibilities
A------ A,B,C,AB,AC,BC,ABC,Φ

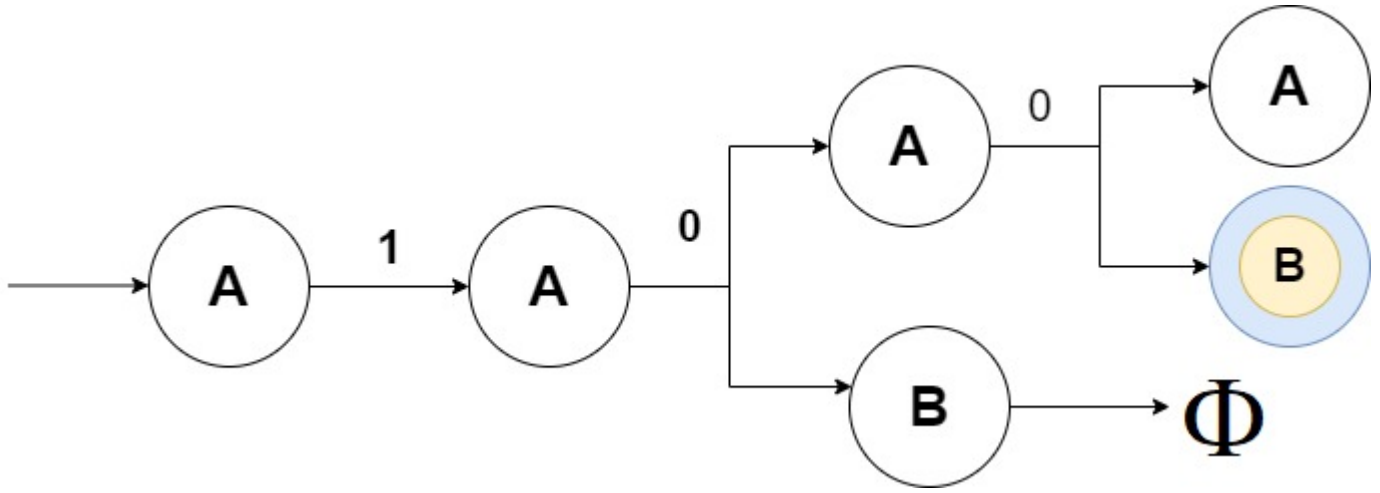
$$\delta = Q \times \sum \rightarrow 2^{Q}$$

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example : 100

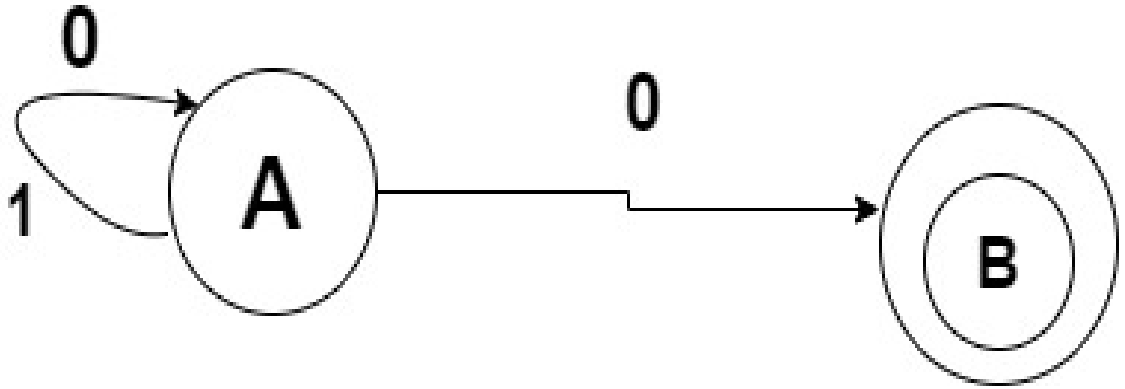


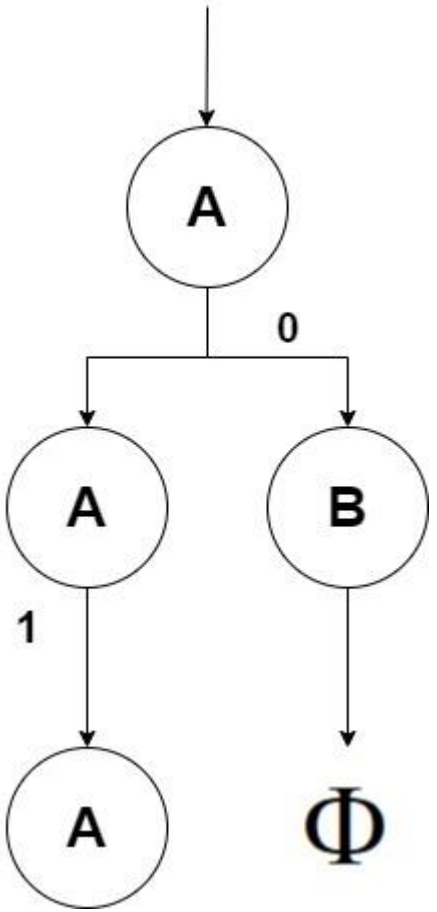
Proliferation Diagram



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

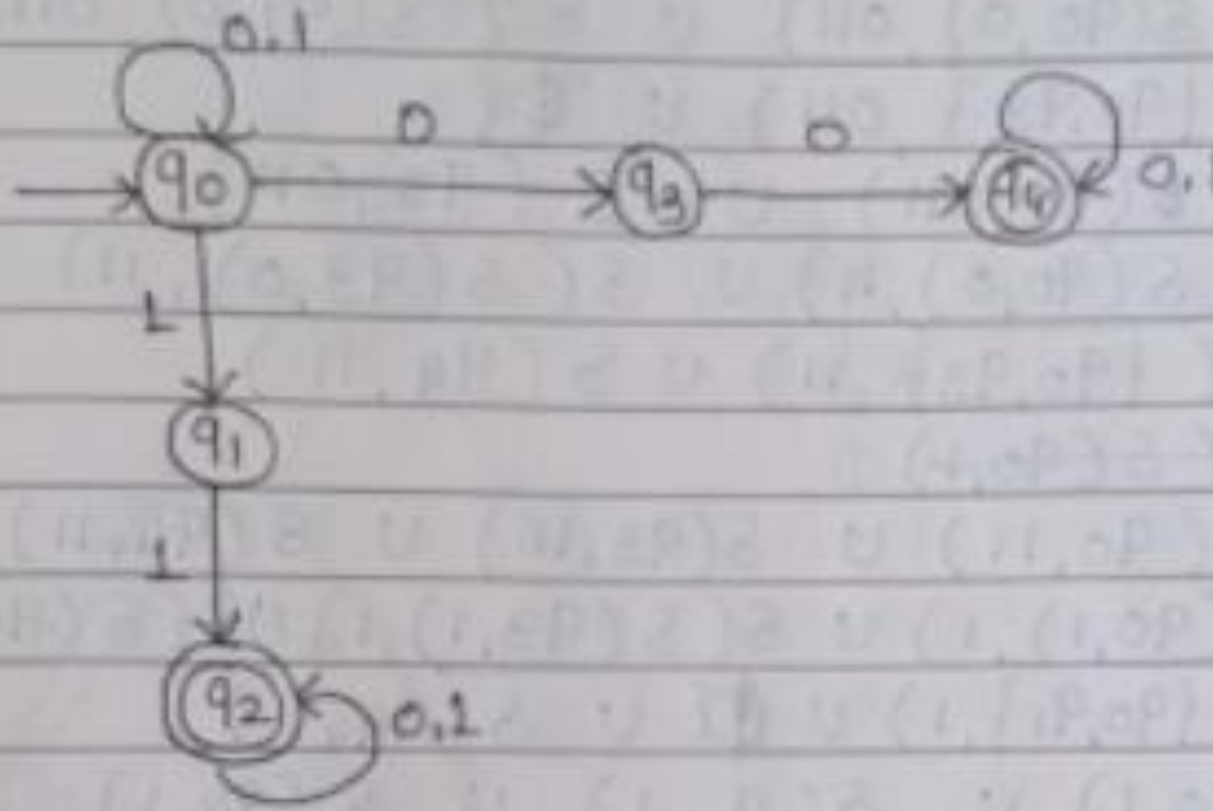
01





Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Q. Design a NFA accepting string with 0's and 1's such that string contains two consecutive 0's and two consecutive 1's.



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

NFA

Ex 1: L1={Set of all strings that ends with ‘1’}

Ex 2: L2= {Set of all strings that contain ‘0’}

Ex 3: L3= {Set of all strings that starts with ’10’ }

Ex 4: L4={Set of all strings that contain ’01’}

DFA

Ex 1: L1={Set of all strings that ends with ‘1’}

Ex 2: L2= {Set of all strings that contain ‘0’}

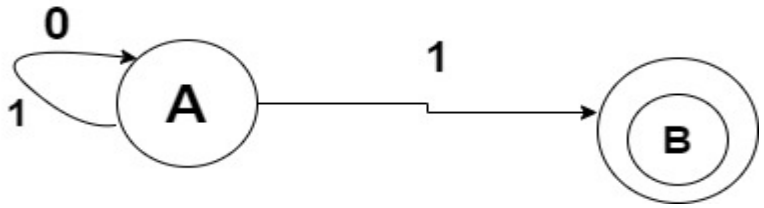
Ex 3: L3= {Set of all strings that starts with ’10’ }

Ex 4: L4={Set of all strings that contain ’01’}

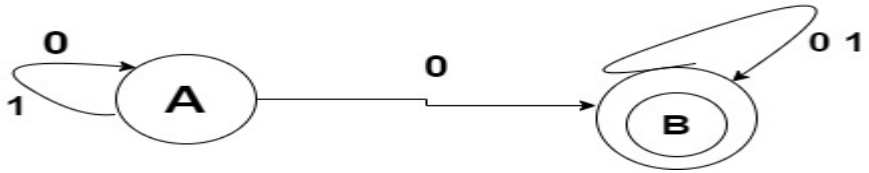
Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

NFA

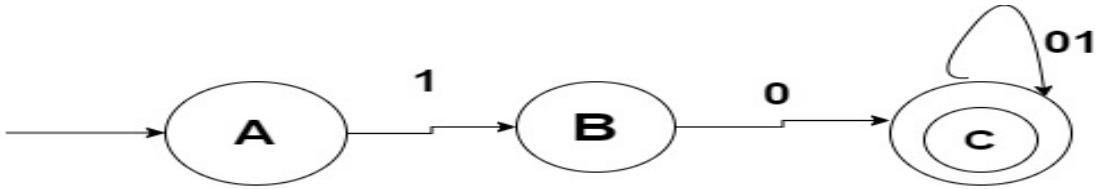
Ex 1: L1={Set of all strings that ends with ‘1’}



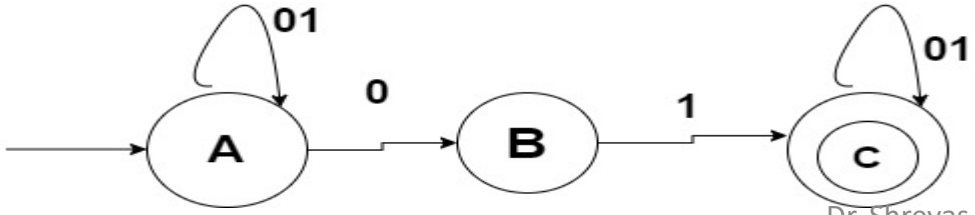
Ex 2: L2= {Set of all strings that contain ‘0’}



Ex 3: L3= {Set of all strings that starts with ’10’ }



Ex 4: L4={Set of all strings that contain ’01’}



DFA

Ex 1: L1={Set of all strings that ends with ‘1’}

Ex 2: L2= {Set of all strings that contain ‘0’}

Ex 3: L3= {Set of all strings that starts with ’10’ }

Ex 4: L4={Set of all strings that contain ’01’}

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Assignment /Home work

Ex 5: L5={Set of all strings that ends with ‘11’}

NFA

DFA

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Conversion of NFA to DFA

Every DFA is an NFA , But not vice versa

But there is an equivalent DFA for every NFA

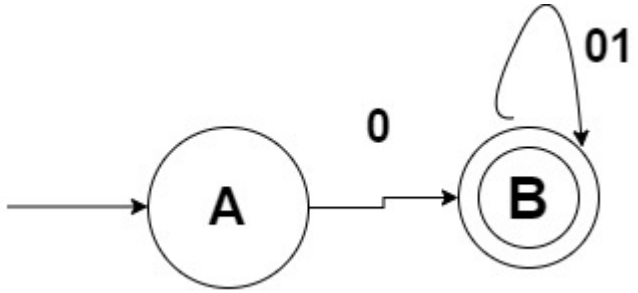
All 5 Tuple are same excepts 1
?????

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

L={Set of all strings over (0,1) that strings with ‘0’}

Σ={0,1}

NFA



State Diagram/ Transition Diagram

State Q	Input 0	Input 1
A	B	Nil
B	B	B

Transition Table

DFA

State Q	Input 0	Input 1
A	B	Nil
B	B	B
C		

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Q. Construct a DFA which accepts the language $L = \{ w \mid w$ has even no. of 0's and odd number of 1's over alphabet $\Sigma = \{0, 1\} \}$

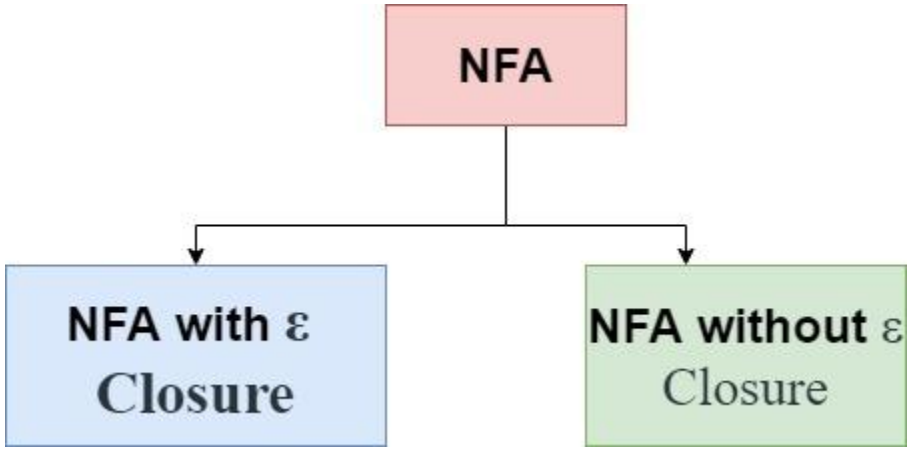
Solution on Page No 53

Q. Construct a DFA which accepts the language $L = \{w \mid w$ has odd number of 0's and even no. 1's over alphabet $\Sigma = \{0, 1\}\}$.

Solution on Page No 54

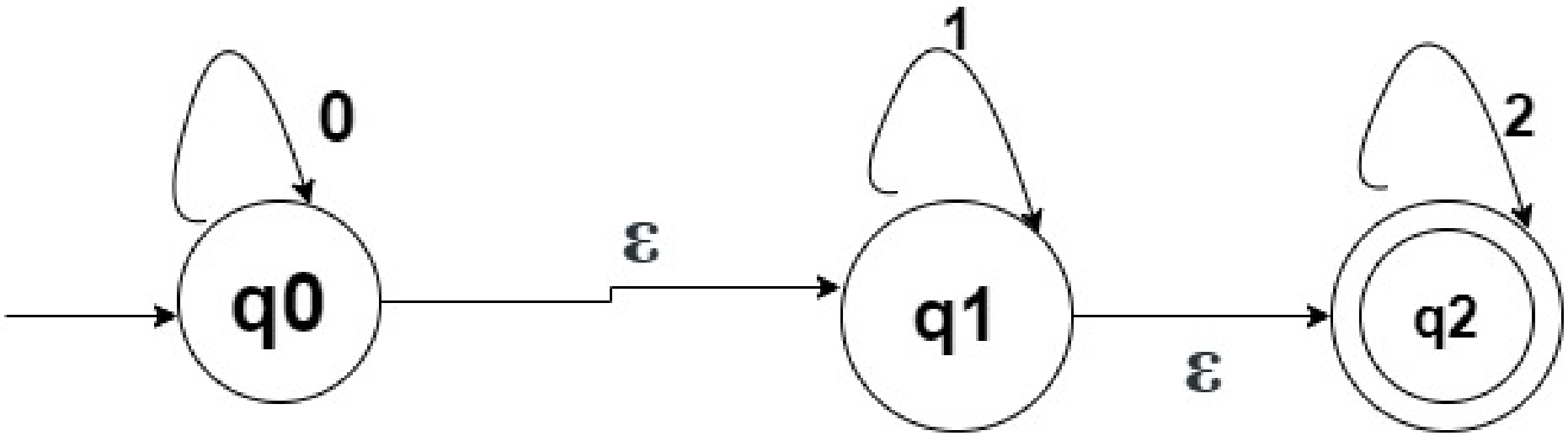
Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda
Sagar University



Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Example: Convert the Following NFA with ε transition into equivalent NFA without ε transition.



Step 1: Find the **ε –Closure of each State**

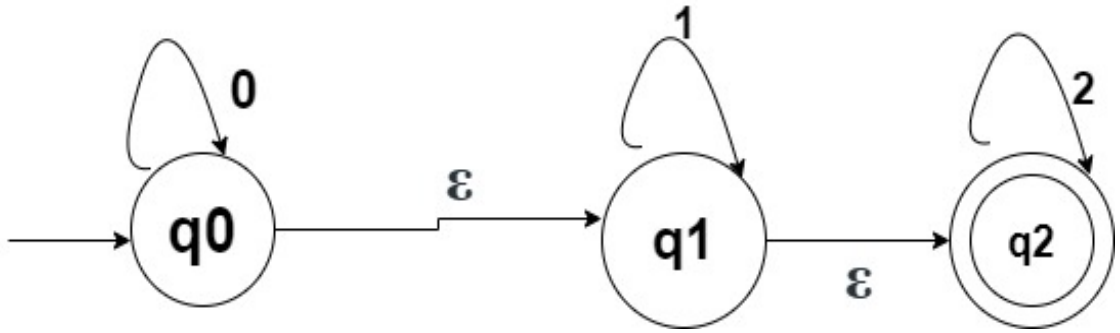
Step 2: Transition Diagram/ Transition Table

Step 3: Transition Function δ'

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Step 1: **ε –Closure of each State**

ε –Closure {qo}=(q0,q1,q2,)
{q1}=(q1,q2,)
{q2}=(q2,)



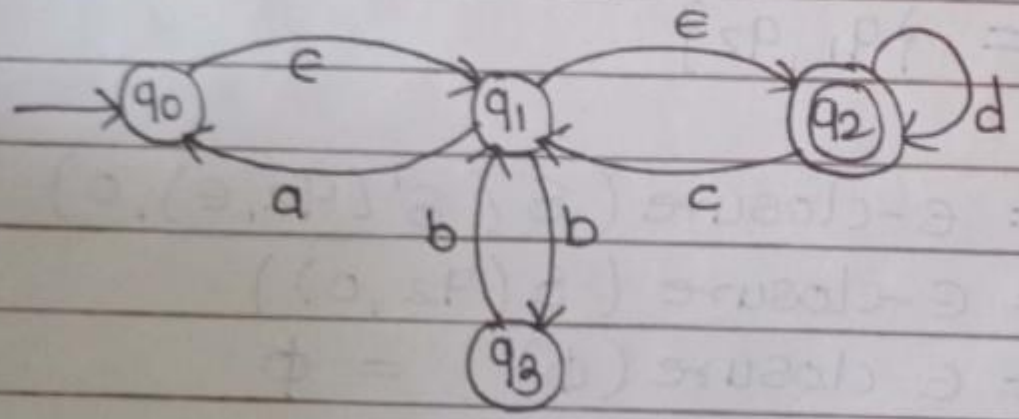
ε –Closure of each state can be find out i,.e
The states which we can visit by ε transition.

Step 2: Transition Table

δ'	0	1	2	ε	Φ
q0					
q1					
q2					

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Q. Convert the following NFA with ϵ- teansition into equivalent NFA without ϵ-teansition.



Solution on Page on 64

Conversion of NFA to DFA

Construct a DFA for the following NFA

M=[{qo,q1},{0,1}, δ,qo,{q1}]

Where, δ is given by

δ (q0,0)={q0,q1}

δ (q0,1)={q1}

δ (q1,0)={ϕ}

δ (q1,1)={q0,q1}

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Q. Construct DFA equivalent to following NFA.

$M = (\{P, q, r, s\}, \{0, 1\}, \delta, P, \{s\})$

where δ is given by.

$\delta(P, 0)$	$= \{P, q\}$	$\delta(q, 0)$	$= \{r\}$
$\delta(P, 1)$	$= \{P\}$	$\delta(q, 1)$	$= \{r\}$
$\delta(r, 0)$	$= \{s\}$	$\delta(s, 0)$	$= \{s\}$
$\delta(r, 1)$	$= \phi$	$\delta(s, 1)$	$= \{s\}$

Answer on Page No.70

Q. Give the steps for conversion of NFA into DFA. Convert the following NFA into equivalent DFA.

$M = (\{P, q, r, s\}, \{0, 1\}, \delta, \{P\}, \{q, s\})$

Where,

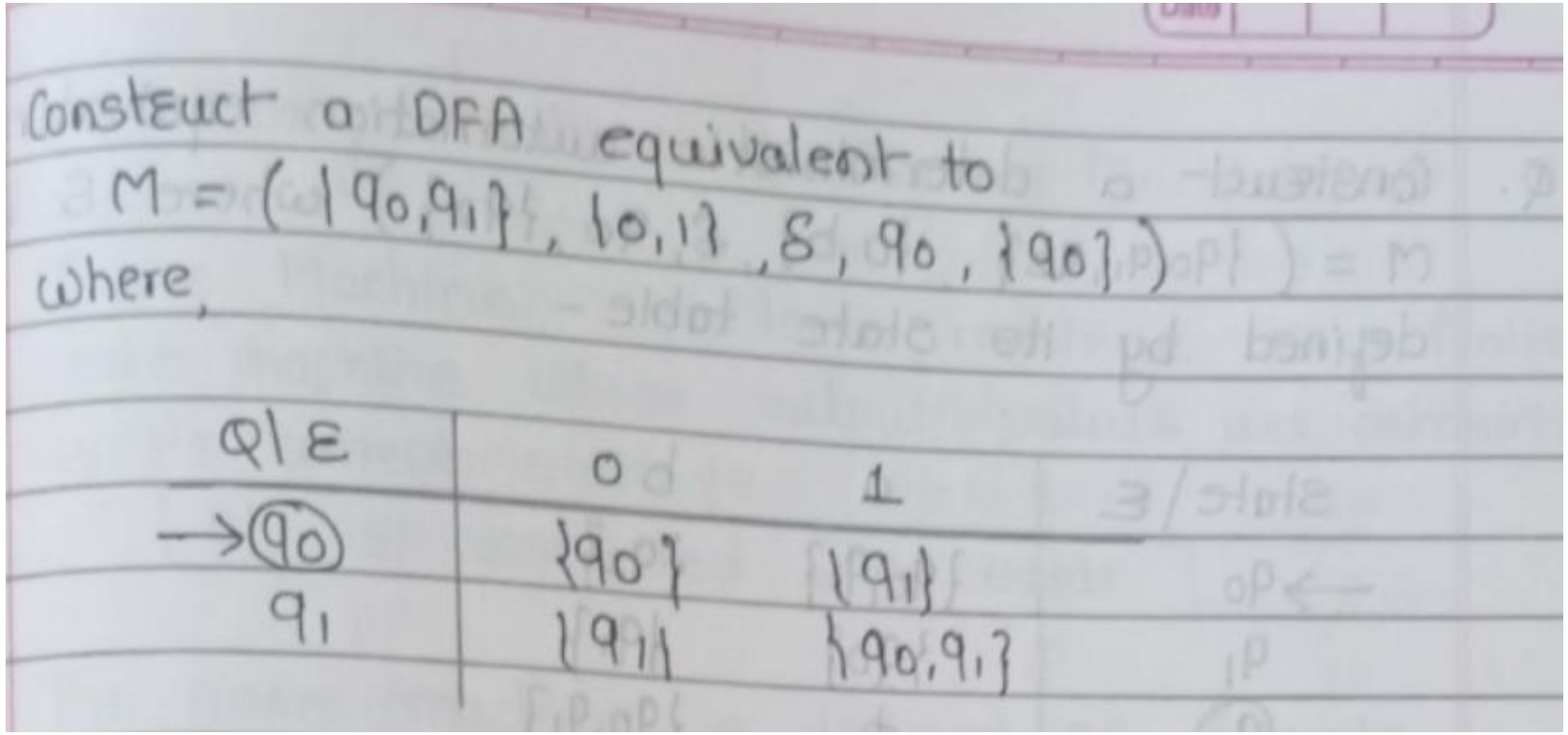
$Q \mid \Sigma$	0	1
$\rightarrow P$	$\{q, s\}$	$\{q\}$
(q)	$\{r\}$	~~$\{P, r\}$~~ $\{q, r\}$
r	$\{s\}$	$\{P\}$
(s)	ϕ	$\{P\}$

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

Q. Convert the following NFA into equivalent DFA $(\{q_0, q_1, q_2, q_3\}, \{0,1\}, \delta, q_0, \{q_3\})$ where δ is

Q \ Σ	0	1
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	$\{q_1\}$
q_2	$\{q_3\}$	$\{q_3\}$
(93)	ϕ	$\{q_2\}$

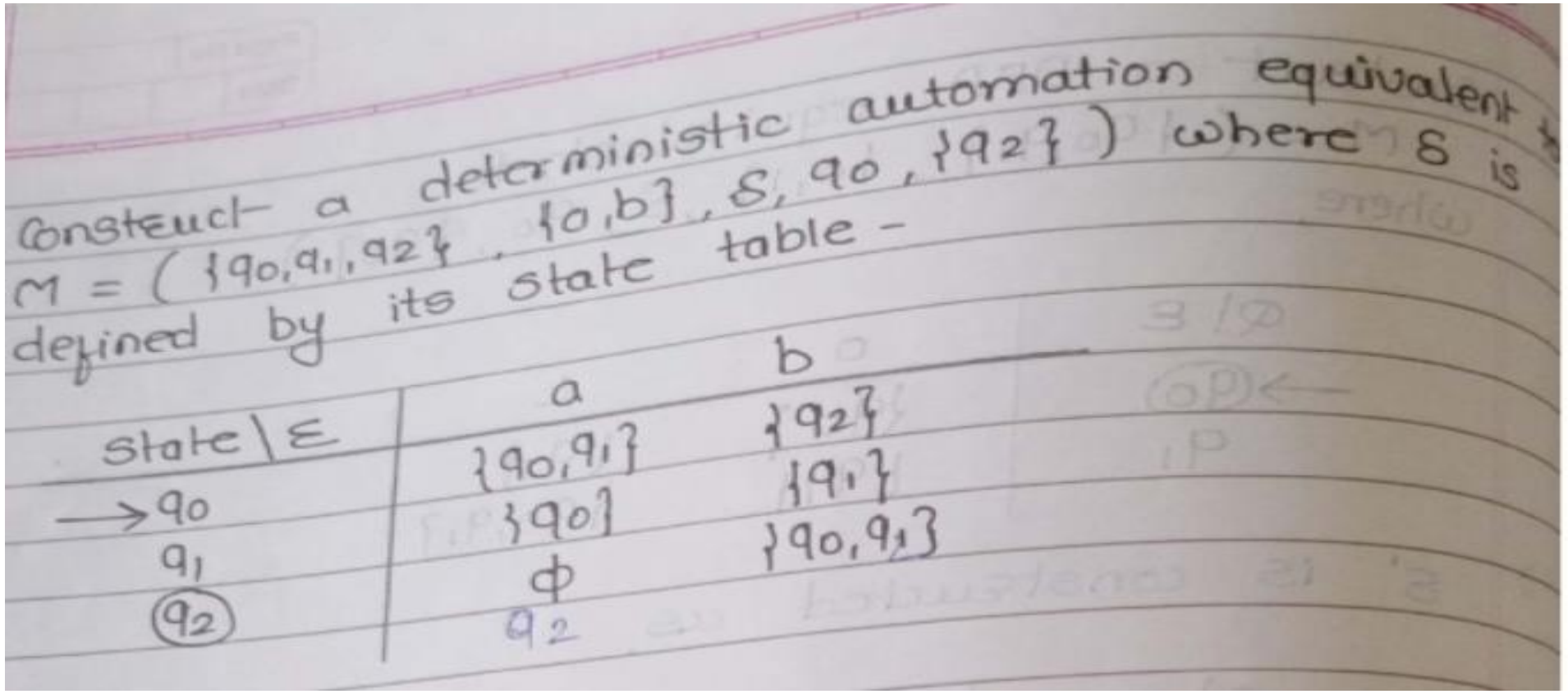
Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University

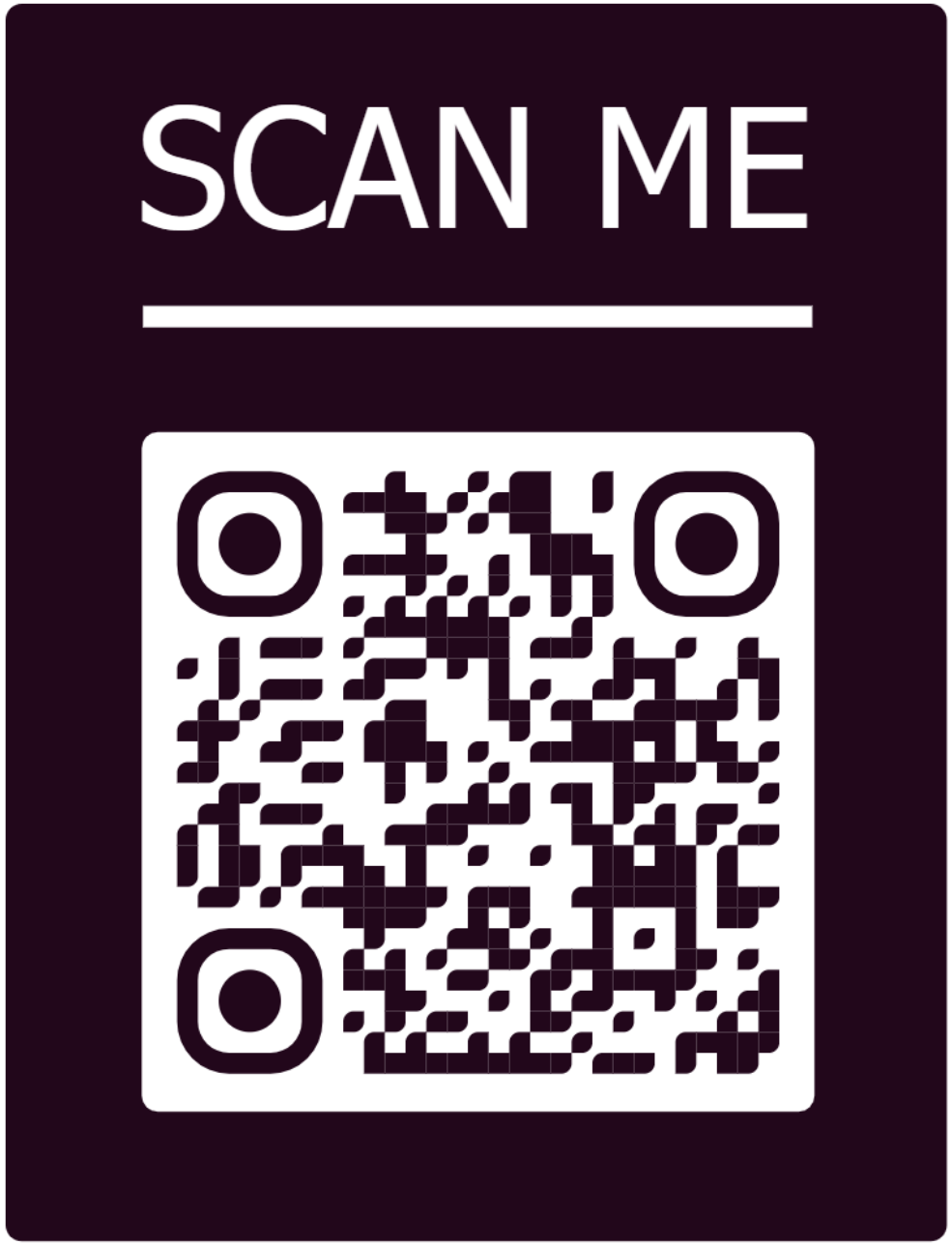

Consteuct a DFA equivalent to
$M = (\{q_0, q_1\}, \{0,1\}, \delta, q_0, \{q_0\})$
where,

$Q \mid \Sigma$	0	1
$\rightarrow (q_0)$	$\{q_0\}$	$\{q_1\}$
q_1	$\{q_1\}$	$\{q_0, q_1\}$

Page No.75

Dr. Shreyas Rajendra Hole, holeysreyas@gmail.com Dayananda Sagar University


Construct a deterministic automation equivalent
M = ({q0, q1, q2}, {a, b}, δ, q0, {q2}) where δ is
defined by its state table -
State \ ε | a | b
→q0 | {q0, q1} | {q2}
q1 | {q0} | {q1}
(q2) | ϕ | {q0, q1}
q2


SCAN ME